NASCERR: Nanosatellite with Electronics Radiation Robust

1st Edilson Filho
Teleinformatics Engineering Department
Federal University of Ceara
Fortaleza, Brazil

2rd Julio Vitorino
Teleinformatics Engineering Department
Federal University of Ceara
Fortaleza, Brazil

3th Carlos Alfredo
Teleinformatics Engineering Department
Federal University of Ceara
Fortaleza, Brazil

4nd Alexandre Almeida
Teleinformatics Engineering Department
Federal University of Ceara
Fortaleza, Brazil

5nd Nícolas de Araújo Moreira
Teleinformatics Engineering Department
Federal University of Ceara
Fortaleza, Brazil

6th Jarbas Silveira
Teleinformatics Engineering Department
Federal University of Ceara
Fortaleza, Brazil

***Abstract*—Faced with increasing access to and demand for space products space products, educational/research institutions and private companies need space devices that are robust and tolerant of faults and space adversities, such as radiation. The NASCERR mission aims to homologate a new version of an on-board computer for CubeSat missions with a strong emphasis on fault tolerance, the ROBC. In addition to redundancy in the power supply and voltage control system, the system also has error correction codes to protect the data in memory, thus offering greater reliability of the data stored by the sensors and stored by the sensors and sent to Earth. The ROBC passed the thermal tests and the error-correcting codes for memory protection were applied to protect the SRAM, bringing greater reliability to the data stored by the sensors. The NASCERR project is is a partnership between universities and the private sector, financed by the Brazilian government**

***Index Terms*—On-board computer, mission satellite, CubeSat, space applications and error-correcting codes**

I. Introduction

Space imposes numerous challenges for space devices, primarily due to the adverse effects of radiation. One of the most significant sources of radiation-induced issues are the Alpha particles, consisting of two protons and two neutrons, which interact with electrons or nuclei of almost every atom they encounter through the Coulombian force [2]. These interactions result in radiation effects such as Total Ionizing Dose (TID) [3] and Single Event Effects (SEE) [4]. Consequently, space missions need systems that exhibit fault-tolerance and are capable of mitigating SEE occurrences. SEEs may occur when deposited charges are collected at sensitive nodes of storage elements (e.g. SRAM cells), can manifest as Hard Errors or Soft Errors [5], [6] and can be silent (occurring over unused or yet used data) or result in a critical fault. Due to this reason, they are a significant concern in the field of space electronics.

When high-energy particles, such as protons or heavy ions, collide with the silicon substrate of a transistor, they disrupt the atoms within the material, releasing energy in the process. This energetic impact excites electrons, leading to the separation of positively charged ions and negatively charged electrons—a phenomenon known as ionization [7]. This disturbance temporarily alters the electronic structure of the transistor.

This disruption can induce a brief but powerful electric current in the transistor, flowing from the substrate to the drain (as illustrated in Figure 1). Such effects are particularly pronounced in sensitive electronic components, like Static Random-Access Memory (SRAM) cells. A direct hit by a high-energy particle on an SRAM cell can trigger a Singular Event (SEE). In this context, a SEE refers to a temporary and unintended alteration in the logical state of the memory cell due to the disturbance introduced by the particle. This momentary shift in the logic state can lead to read or write errors in the data stored within the SRAM cell, underscoring the critical importance of addressing these radiation-induced effects in space electronics.

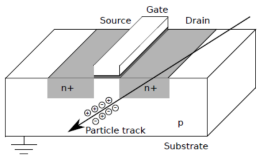


Fig. 1. Effect of the collision of an energetically charged particle on a transistor. Source [7].

The Figure 2 shows the effect of an SEE on a transistor, we can see the close-up of a transistor hit by a Single-Event Effect.

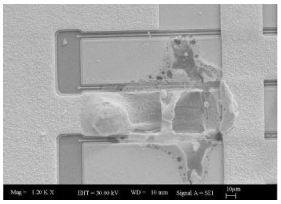


Fig. 2. Effect of an ESS on a transistor. Source [9].

Understanding the effects of high-energy particles on electronic devices is essential for the design of robust systems, especially in space applications and in environments where exposure to radiation is a concern. Mitigation strategies, such as the use of radiation-tolerant design techniques, are employed to minimize the impact of these unique events on the normal operations of electronic devices in hostile environments.

The effects of charged particles are more significant in Low Earth Orbit (LEO) due to the proximity of satellites to Earth's atmosphere and the Van Allen radiation belts. LEO, which extends up to approximately 2,000 kilometers above the Earth's surface, is within the reach of the inner Van Allen radiation belt [8]. This belt is characterized by a dense concentration of charged particles, including protons and electrons, trapped by Earth's magnetic field. These particles impose a substantial threat to electronic components onboard satellites in LEO.

Given this scenario and considering that most nanosatellite missions use LEO orbit, the study of the effects of radiation on electronic components and the production of electronic devices with an emphasis on fault-tolerance is extremely important. This work presents the NASCERR[1] mission, a CubeSat nanosatellite that provides a hardware (on-board computer) and firmware (error-correcting codes) solution for nanosatellite space missions. The main contributions of this paper are: (1) Provide a new on-board computer architecture for CubeSat missions with an emphasis on fault tolerance and; (2)Implement new in-memory error-correcting code techniques for CubeSat missions.

II. Prelimninaries

This section is divided into: *Cubesats* and *Related work*

A. Cubesat

The advances in Very Large Scale integration (VLSI) technology led to the miniaturization of complex systems, complete complex systems can be built into a single silicon chip called System-on-Chip (SoC), which contributed to the development of nanosatellites [10].

CubeSats, short for Cube Satellites, are miniature satellites with standardized designs that have revolutionized space exploration. space exploration. These small, cube-shaped satellites are divided into units, each of which is a standard 10 centimeters on each side, with a total volume of 1 liter, and weigh no more than 1.33 kilograms [11]. Their compact size and low weight make them economical and accessible to institutions educational institutions, research organizations and commercial entities and commercial entities, democratizing access to space. Despite their size, they are so complex that have many of the characteristics of large satellites and are still exposed to vacuum effects, extreme thermal amplitudes and the effects of of ionizing radiation [10].

The use of these small satellites has allowed new space missions to be launched at lower costs.

[1]Nanosatellite with Electronics Radiation Robust

B. Related Works

Some articles had some highlights and we will present them in Table I.

TABLE I
List of articles and works related to the topic

Paper	Advantages	Drawbacks
[12]	-The article describes a fault-resistant electrical system design for CubeSats. The aim is to minimize stress on components, extend service life and incorporate hardware redundancy. An interesting feature is the implementation of independent maximum power point converters for each solar panel, overcoming space limitations in smaller CubeSats.	Although the paper focuses on proposing a new EPS architecture for Cubesats that improves fault tolerance capabilities, it does not address techniques for effective recovery from failures.
[13]	-The article proposes improvements in software validation for CubeSats, with an emphasis on space sensitivity through fault injection. It highlights the need to integrate fault tolerance techniques into the development process, in order to deal with transient processor faults and ensure fault tolerance due to space radiation.	Even though the article addresses fault tolerance techniques as part of its general development approach, it does not directly address techniques that can effectively prevent errors, such as Single Event Effect (SEE).
[14]	-The article introduces a Multiprocessor System on a Chip (MPSoC) with high fault tolerance. The MPSoC is based on the ARM Cortex-A53 processor and employs a three-step approach to fault mitigation, which enables fault detection and correction, reconfiguration of faulty units and reallocation of resources, ensuring reliability in critical applications such as satellites.	This paper presents a hybrid hardware and software approach that enables fault tolerance and FPGA reconfigurability and mixed thread-level criticality. However, it is not clear which applications are used to correct errors.

Given the challenges presented by the hostile space environment for electronic devices, especially On-Board Computers (OBCs), it is vital to consider the possible fragility of CubeSats in relation to space radiation, which has represented a significant threat, causing failures and degradation. Therefore, it is crucial to continue improving solutions that guarantee fault tolerance in onboard systems used in space missions, taking into account the dynamic and challenging nature of space. In this sense, NASCERR aims to approve a new architecture with fault tolerance implementations at the hardware (HW) and firmware (FW) levels.

III. Nascerr: Nanosatellite with Electronics Radiation Robust

NASCERR is a mission that aims to be a space laboratory emphasizing fault tolerance techniques for space missions. The ROBC on-board computer is a hardware solution and the error-correcting code for in-memory protection is a firmware solution.

This section is divided into: NASCERR's mission and system engineering, on-board computer architecture, communication protocols and payload.

A. Mission

The main objective of the NASCERR project is to homologate a new on-board computer with an emphasis on fault

tolerance that meets the CubeSat standard, ROBC (Robust On-board Computer). The nanosatellite developed is a 2U Cubesat: the first unit is a complete CubeSat and the second unit contains the mission payload, which is the ROBC on-board computer.

In space systems engineering, the lifecycle of a satellite [15] refers to the stages it goes through, from conceptualization to design, development, launch, operation, and eventual disposal. This systematic approach ensures that satellites are effectively planned, constructed, and utilized throughout their mission duration. See Figure 3 presents the design of the NASCERR life cycle process.

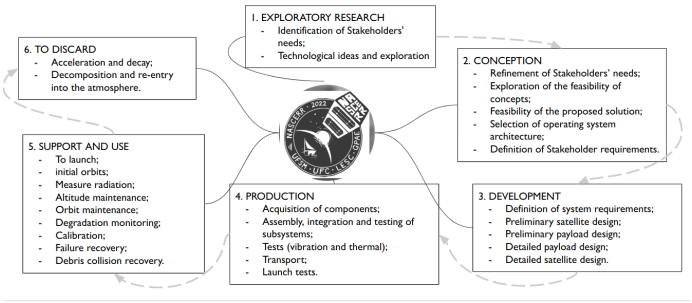


Fig. 3. Lifecycle Process Scenario. Author.

The diagram in Figure 4 shows some of the main modules of the NASCERR CubeSat. The TC/TM is responsible for receiving and sending commands from/to the ground station. The Onboard Computer that will be used in the CubeSat's 1U control using the On-board computer (OBC) from ISIS[2]. The MGE and MDCA Modules are respectively the Gravity and Space Geodesy Module and the Magnetic Attitude Control Module. Finally, in the 1U are the payloads, ROBC and error-correction codes for in-memory protection.

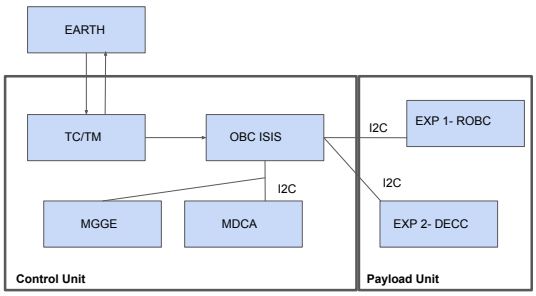


Fig. 4. Diagram NASCERR. Author.

The payload unit has two main functions: The testing of ROBC, the main payload of the mission, and the testing error-correction codes for in-memory protection. The following subsection discusses some of the features of this OBC.

[2]https://www.isispace.nl/product/on-board-computer/

B. Payload Unit: ROBC architecture

A quick presentation of the ROBC architecture is shown in Figure 5. For example, it is possible to see the CAN[3], MCU, SRAM, Power Source and Redundancy and the Current Sensor.

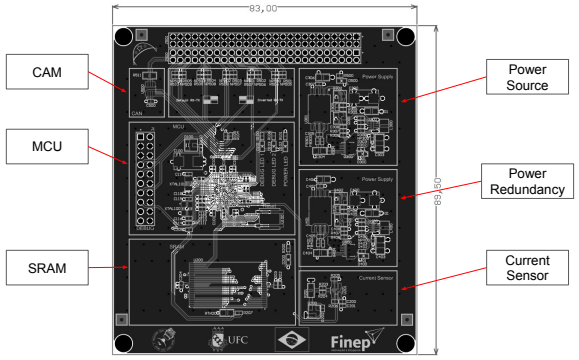


Fig. 5. Architectute ROBC. Author.

1) Bill of Materials: The Table II lists a short version of Bill of Materials used for the assembly of OBC.

TABLE II
SOME ITEMS OF BOM

Item	Description
STM32L4S5QII6	IC MCU 32BIT 2MB FLASH 132UFBGA PN: STM32L4S5QII6
IS61WV204816BLL-10TLI	IC SRAM 32MBIT 48TSOP PN: IS61WV204816BLL-10TLI
ADUM5028-3BRIZ	IC ISOL DC/DC CONVERTER 3.3V 300MW PN: ADUM5028-3BRIZ
ECS-80-12-33-JGN-TR	CRYSTAL 8.0000MHZ 12PF SMD PN: ECS-80-12-33-JGN-TR
ECX-.327-CDX-1293	CRYSTAL 32.768kHZ 12.5pF SMD
SIS407DN-T1-GE3	MOSFET P-CH 20V 25A 9.5mOhm PPAK1212-8 PN: SIS407DN-T1-GE3

2) Power Redundancy : The implementation of a power redundancy system on CubeSats is essential to ensure the continuity of space operations and the integrity of missions [19]. In a space environment where failures are common due to various reasons, from weather conditions to equipment malfunctions, power redundancy ensures that the CubeSat remains operational even when a power source fails. This not only protects the valuable scientific instruments on board, but also significantly extends the lifespan of the mission, enabling continuous data collection and the execution of experiments for longer periods. Furthermore, by integrating multiple power sources, such as solar panels and batteries, CubeSats can adapt to various conditions, such as temporary solar eclipses, ensuring that the batteries are recharged and the CubeSat remains operational, thus contributing to the success of the space mission.

C. Payload Unit: Error Correcting Codes

Error Correcting Codes (ECC) are extremely important error detection and correction techniques in telecommunications systems, space applications, aviation and others. Over the years, various papers have presented fundamental concepts for

[3]CAN is a serial communications protocol that allows distributed control in real-time, with a high level of security

the development and implementation of these algorithms [20] and [21].

One of NASCERR's objectives is to implement new ECC techniques for memory protection in a CubeSat. Some of the algorithms are: CLC [22] and TBEC-RSC [23].

1) CLC: The column-line code (CLC) is inspired by the Hamming code and parity bits. It is designed to correct multiple memory errors, in particular typical error patterns resulting from radiation effects on memory content, especially in space applications. The main motivation for implementing CLC is the lower overhead in terms of area and power compared to Reed-Muller[4], while maintaining higher correction rates than those offered by Matrix. CLC is applied in data information consisting of 16 bits, and has its codified word consisting of 40 bits, whereby there are 16 bits of word (C), 8 bits of parity (P), 4 bits of word parity (Pa), and 12 check bits (CB). See Figure 6.

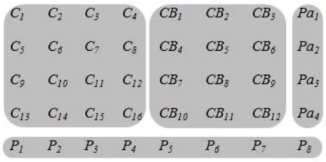


Fig. 6. Codified CLC word's model. Source [22].

The Figure 7 shows a layout of how ECC is instantiated in a device's firmware.

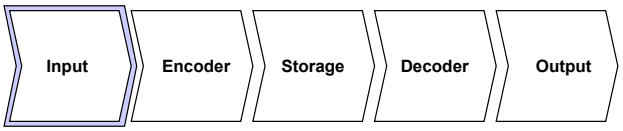


Fig. 7. General scheme for ECC instantiation. Author.

D. Protocols

The experiments will use the OBC to access a peripheral SRAM memory to record their results. A communication protocol was established to allow the experiments to send requests to the OBC and receive the appropriate response to accomplish this task.

Algorithm 1 illustrates how the OBC processes the received messages. The memory queue used is unique for each experiment and they are associated with the experiment ID. The write and read commands rely on communication with the SRAM and can have communication failures. The used_space and free_space commands are processed internally on the microprocessor by accessing the memory queue object so they aren't susceptible to communication errors.

The memory management is done using a ring queue abstraction. Each experiment has its own part of the SRAM with a ring queue associated with it. The queue has a head pointer, referencing the address of the oldest data written, and a tail address, referencing the next available memory slot.

When an experiment requests the OBC to write data on the memory, the OBC writes the data on the memory address pointed by the tail of the queue and then moves the tail to the next memory address. When the queue is full the tail and the header are pointing to the same address. In case the OBC needs to write data into a full queue, it will overwrite the data on the address and move both the tail and the header to the next memory address.

When an experiment requests the OBC to read data from the memory, the OBC reads the data from the memory address pointed by the head of the queue and then moves the head to the next memory address. When the OBC reads the last data available data it sets the queue to an empty state until it receives a write command. While in this empty state all read requests will return that the queue is empty.

The communication is done using Universal Synchronous and Asynchronous Receiver-Transmitter (USART), Inter-Integrated Circuit(I^2C), or Controller Area Network (CAN).

The message frame used consists of 5 bytes: an identification byte, a command byte, two data bytes and a checksum byte. It can be visualized in the Figure 8.

Algorithm 1 OBC request processing

Data: $MessageTypeDefmsg, MemoryQueueTypeDefmem_queue$
Result: $MessageTypeDefresponse_msg$
switch *msg.Command* **do**
  case *WRITE* **do**
    response $\leftarrow mem_queue.write(msg.Data)$
    if *response is SUCESSFUL* **then**
      response_msg.Command $\leftarrow SUCESSFUL_WRITE$
      _
    else
      response_msg.Command $\leftarrow FAILED_WRITE$
      _
    response_msg.Data $\leftarrow 0$
    _
  case *READ* **do**
    response, data $\leftarrow mem_queue.read()$
    if *response is SUCESSFUL* **then**
      response_msg.Command $\leftarrow SUCESSFUL_READ$
      $_response_msg.Data \leftarrow data$
    else
      response_msg.Command $\leftarrow FAILED_READ$
      $response_msg.Data \leftarrow 0$
      _
  case *USED_SPACE* **do**
    used_space $\leftarrow mem_queue.used_space()$
    $response_msg.Command \leftarrow USED_SPACE_OK$
    $response_msg.Data \leftarrow used_space$
    _
  case *FREE_SPACE* **do**
    free_space $\leftarrow mem_queue.free_space()$
    $response_msg.Command \leftarrow FREE_SPACE_OK$
    $response_msg.Data \leftarrow free_space$
    _
  otherwise do
    response_msg.Command $\leftarrow INVALID_COMMAND$
    $response_msg.Data \leftarrow 0$
    _
  return *response_msg*

[4]Reed-Muller codes are error correction codes that are used in wireless communication applications, particularly in deep space communication

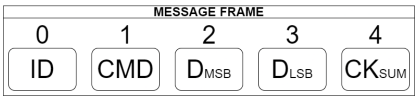


Fig. 8. Message frame used in the protocol. Author.

The identification(ID) byte will be used by the OBC to distinguish between the different experiments.

The second byte will contain the command code in the request message and the response code in the returning message. Table III shows the possible commands and their respective values. The TEST command is used for debug operations and has no practical use. The WRITE command will order the onboard computer to write the data on the memory queue of the experiment. The READ command will read the oldest data on the queue of the experiment and send it back on a response message. The USED_SPACE command will return to the experiment the occupied memory space on the queue and the FREE_SPACE will return the free memory space on the queue.

TABLE III
COMMANDS AND THEIR RESPECTIVE VALUES

Command	Value
TEST	0
WRITE	1
READ	2
USED_SPACE	3
FREE_SPACE	4

IV. RESULTS

A. Thermal Analyses

Thermal analysis tests are crucial for space devices in low earth orbit (LEO) due to the extreme temperature variations experienced in space. In LEO, space devices are subject to significant temperature fluctuations as they move in and out of sunlight and the Earth's shadow. These temperature changes can range from extremely hot when exposed to direct sunlight to extremely cold in the absence of sunlight. Figure 9 shows the results of the thermal test of the ROBC board during the data encoding and decoding operation. The result of the physical experiment of the temperature test shows that, during system start-up, the maximum temperature was 26.5ºC. This result is interesting because CubeSats, like any space device, need to have optimal predictability and thermal control. In general, the thermal control on CubeSats is passive, with heat sinks and optical tape on the external structure, keeping the structure, with the exception of the solar panels, in a range of approximately -15 °C to 40 °C for sun-synchronous orbits. [18]

B. Error-correcting codes

For the error injection test, we added an algorithm that reads the data from memory storage, randomly inverts it from 0 to 8 bits and then rewrites the data in memory. After this process,

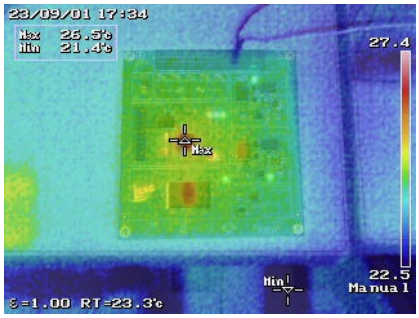


Fig. 9. Thermal Analyses. Author.

the decoder is activated and an analysis module checks and counts the efficiency of the code. The Figure 10 shows when the errors were inserted. For error injection, we developed an algorithm that changed from 0 to 8 bits of the stored data. After this inversion, we used the ECC decoder to check the stored data.

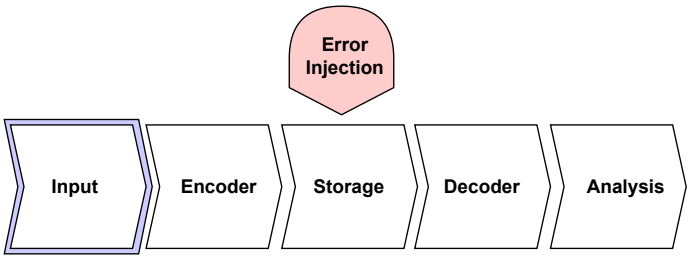


Fig. 10. Error Injection. Author.

The algorithm was tested in batches of 10000. A Python script was used to send random data to the OBC through UART communication. The OBC then stored the data in its memory using the ECC encoding. After the script receives the response message from the OBC indicating that the writing was successful, it sends a read request. The OBC would then read the encoded data and add an error to it. Afterwards, it would decode while trying to correct the random error. The OBC then sends the decoded data to the script. Finally, the script compares the received data with the starting data and determines if the algorithm successfully corrected the error.

Two kinds of errors were tested, a localized error and a random error. The localized error is created by randomly selecting a continuous memory space and flipping all of its bits. The random error is created by randomly selecting individual memory space and flipping each of the selected bits.

In Figure 11 it can be observed that even amounts of localized error perform better than the equivalent amount of random error. We chose to implement tests with two types of errors just to check the ability of the correct code to perform the error correction process. Adjacent errors are more likely to happen because when the ionized particle hits a memory region, it hits an area and has a better chance of influencing the neighborhood [22].

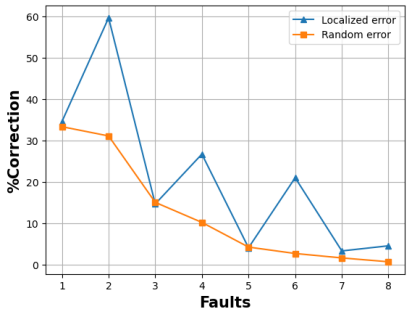


Fig. 11. Comparison between localized error and random error.

The NASCERR Mission also aims to be a test laboratory in space, so new ECCs will be implemented and tested. In this way, new error correction algorithms can be transmitted to the satellite via the uplink.

V. Conclusion

In this document, a detailed analysis of the CubeSat mission NASCERR, along with a presentation of some of the fault tolerance techniques that have been implemented, such as the use of power resource redundancy and error-correcting codes for in-memory protection. This study provides the reader with a good understanding of the importance of implementing protection strategies in firmware and hardware to mitigate the adverse effects that space weather can have on electronic systems in space.

The CLC code implemented to protect the SRAM memory uses 4.02Kb, which is less than 0.01% of the available memory, leaving more than 98% of the memory available for other applications and data storage.

The thermal test of the ROBC board in operation met the expected requirement, falling short of the manufacturer's specification. It is clear that within the structure of the complete CubeSat other elements will radiate heat and the thermal dispersers will act. In a future work, we intend to complete the structural, thermal and vibration tests involving the entire structure of the NASCERR CubeSat. Other error correction algorithms [23] are also being studied for future implementation.

Acknowledgments

Part of the results presented in this article were carried out within the scope of the project "NASCERR" which is funded by Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), call for project CNPq/MCTI/FNDCT 19/2020 Desenvolvimento de CT&I para Cubesats e suas Aplicações às Áreas de Interesse do Brasil, and "ROBC" wichs is funded by Financiadora de Estudos e Projetos (FINEP), call CH PÚB MCTI/FINEP/AT Materiais Avançados E Minerais Estratégicos 2020. The authors thank the contribution of the research team who provided feedback on this study and supported this work.

References

[1] Space, Alen. "A basic guide to nanosatellites." Alen Space (2020).
[2] FONSECA, Zamorano, Luna, Santos. Estudo do poder de frenamento de partículas alfa na matéria, Revista Brasileira de Ensino de Física, v.32, n.2, 2305 (2010).
[3] BARNABY, H. J. Total-ionizing-dose effects in modern CMOS technologies. IEEE Transactions on Nuclear Science, v. 53, n. 6, p. 3103-3121, 2006.
[4] WELLER, Robert A. et al. Monte Carlo simulation of single event effects. IEEE Transactions on Nuclear Science, v. 57, n. 4, p. 1726-1746, 2010.
[5] HAQUE, Imran S.; PANDE, Vijay S. Hard data on soft errors: A large-scale assessment of real-world error rates in GPGPU. In: 2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing. IEEE, 2010. p. 691-696.
[6] ESA-ESTEC Data Systems Division / Microelectronics Section. Space Engineering, Product Assurance: Techniques for Radiation Effects Mitigation in ASICs and FPGAs. Noordwijk, The Netherlands, 2009.
[7] MCCARTNEY, Mark P. SRAM reliability improvement using ECC and circuit techniques. 2014. Tese de Doutorado. Carnegie Mellon University.
[8] CHEN, Yue; REEVES, Geoffrey D.; FRIEDEL, Reiner HW. The energization of relativistic electrons in the outer Van Allen radiation belt. Nature Physics, v. 3, n. 9, p. 614-617, 2007.
[9] https://www.esa.int/ESA_Multimedia/Images/2011/06/Catastrophic_latch-up_due_to_heavy_ion. Access: October 18th, 2023.
[10] FORTESCUE, Peter; STARK, John and SWINERD, Graham. Spacecraft systems engineering, 3rd edition. John Willey and sons, 2003.
[11] XUE, Yong et al. Small satellite remote sensing and applications–history, current and future. International journal of remote sensing, v. 29, n. 15, p. 4339-4372, 2008.
[12] A. Edpuganti, V. Khadkikar, M. S. Elmoursi, H. Zeineldin and M. Al Hosani, "A Novel EPS Architecture for 1U/2U Cubesats with Enhanced Fault-Tolerant Capability," 2020 IEEE Industry Applications Society Annual Meeting, Detroit, MI, USA, 2020, pp. 1-6, doi: 10.1109/IAS44978.2020.9334817.
[13] D. Paiva, R. Lima, M. Carvalho, F. Mattiello-Francisco and H. Madeira, "Enhanced software development process for CubeSats to cope with space radiation faults," 2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC), Beijing, China, 2022, pp. 78-88, doi: 10.1109/PRDC55274.2022.00022.
[14] C. M. Fuchs, N. M. Murillo, A. Plaat, E. van der Kouwe and P. Wang, "Towards Affordable Fault-Tolerant Nanosatellite Computing with Commodity Hardware," 2018 IEEE 27th Asian Test Symposium (ATS), Hefei, China, 2018, pp. 127-132, doi: 10.1109/ATS.2018.00034.
[15] FISCHER, Philipp M. et al. Implementing model-based system engineering for the whole lifecycle of a spacecraft. CEAS Space journal, v. 9, n. 3, p. 351-365, 2017.
[16] ARORA, D., Gautham, S., Gupta, H., and Bhushan, B. (2019, October). Blockchain-based security solutions to preserve data privacy and integrity. In 2019 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS) (pp. 468-472). IEEE.
[17] LIU, Tianyu et al. Degradation modeling of satellite thermal control coatings in a low earth orbit environment. Solar Energy, v. 139, p. 467-474, 2016.
[18] SCHOLZ, Artur et al. Flight results of the COMPASS-1 picosatellite mission. Acta Astronautica, v. 67, n. 9-10, p. 1289-1298, 2010.
[19] YE, Zeyu et al. Fault reconstruction method of high redundancy satellite power distribution unit. IET Power Electronics, 2023.
[20] HAMMING, Richard W. Error detecting and error correcting codes. The Bell system technical journal, v. 29, n. 2, p. 147-160, 1950.
[21] PETERSON, William Wesley; WELDON, Edward J. Error-correcting codes. MIT press, 1972.
[22] H. d. S. Castro, J. A. N. da Silveira, A. A. P. Coelho, F. G. A. e Silva, P. d. S. Magalhães and O. A. de Lima, "A correction code for multiple cells upsets in memory devices for space applications," 2016 14th IEEE International New Circuits and Systems Conference (NEWCAS), Vancouver, BC, Canada, 2016, pp. 1-4, doi: 10.1109/NEWCAS.2016.7604783.
[23] F. Silva, A. C. Pinheiro, J. A. N. Silveira and C. Marcon, "A Triple Burst Error Correction Based on Region Selection Code," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 31, no. 8, pp. 1214-1222, Aug. 2023, doi: 10.1109/TVLSI.2023.3273085.
[24] SILVA, Felipe et al. Extended matrix region selection code: An ECC for adjacent multiple cell upset in memory arrays. Microelectronics Reliability, v. 106, p. 113582, 2020.